mc_control_alpha

October 12, 2026

[2]:
```
import os, sys;
sys.path.append(os.path.abspath('..')) # for importing the parent dirs
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from common.gridworld import GridWorld
# from common.utils import greedy_probs


def greedy_probs(Q, state, epsilon=0, action_size=4):
    qs = [Q[(state, action)] for action in range(action_size)]
    max_action = np.argmax(qs)

    base_prob = epsilon / action_size
    action_probs = {action: base_prob for action in range(action_size)}  #{0: /
 ↪4, 1: /4, 2: /4, 3: /4}
    action_probs[max_action] += (1 - epsilon)
    return action_probs


class McAgent:
    def __init__(self, epsilon=0.1, alpha=0.1):
        self.gamma = 0.9
        self.epsilon = epsilon  # (     ) -
        self.alpha = alpha    # (     ) Q
        self.action_size = 4

        random_actions = {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
        self.pi = defaultdict(lambda: random_actions)
        self.Q = defaultdict(lambda: 0)
        # self.cnts = defaultdict(lambda: 0)
        self.memory = []

    def get_action(self, state):
        action_probs = self.pi[state]
        actions = list(action_probs.keys())
        probs = list(action_probs.values())
```

```
        return np.random.choice(actions, p=probs)

    def add(self, state, action, reward):
        data = (state, action, reward)
        self.memory.append(data)

    def reset(self):
        self.memory.clear()

    def update(self):
        G = 0.0
        total_q_delta = 0.0
        greedy_changes = 0
        for data in reversed(self.memory):
            state, action, reward = data
            G = self.gamma * G + reward
            key = (state, action)
            # Monte Carlo Control:    Q
            old_q = self.Q[key]
            self.Q[key] += (G - old_q) * self.alpha
            total_q_delta += abs(self.Q[key] - old_q)

            #
            old_greedy = max(self.pi[state], key=self.pi[state].get)
            self.pi[state] = greedy_probs(self.Q, state, self.epsilon)
            new_greedy = max(self.pi[state], key=self.pi[state].get)
            if old_greedy != new_greedy:
                greedy_changes += 1

        #    Return       G         G0
        mean_q_delta = total_q_delta / len(self.memory) if self.memory else 0.0
        return G, mean_q_delta, greedy_changes



def moving_average(values, window=200):
    #   Episode         window
    values = np.asarray(values, dtype=float)
    if len(values) < window:
        window = len(values)
    if window == 0:
        return np.array([]), np.array([])
    averaged = np.convolve(values, np.ones(window) / window, mode='valid')
    indices = np.arange(window, len(values) + 1)
    return indices, averaged
```

```
def plot_learning_curves(histories, parameter_name, parameter_values,␣
 ↪window=200):
    fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)
    for value in parameter_values:
        hist = histories[value]
        x, return_avg = moving_average(hist['returns'], window)
        _, delta_avg = moving_average(hist['q_delta'], window)
        axes[0].plot(x, return_avg, label=f'{parameter_name}={value}')
        axes[1].plot(x, delta_avg, label=f'{parameter_name}={value}')

    axes[0].set_title('Episode Return (discounted G0)')
    axes[0].set_ylabel('Moving average of G0')
    axes[1].set_title('Mean absolute Q update per episode')
    axes[1].set_ylabel('Moving average of mean |dQ|')
    axes[1].set_xlabel('Episode')
    for ax in axes:
        ax.grid(alpha=0.3)
        ax.legend()
    fig.tight_layout()
    plt.show()


def summarize_q_change(snapshot, final_q):
    #          State-Action      0
    keys = set(snapshot) | set(final_q)
    if not keys:
        return 0.0, 0
    mean_change = np.mean([abs(final_q.get(k, 0.0) - snapshot.get(k, 0.0))
                           for k in keys])
    return float(mean_change), len(keys)

alpha_values = [0.01, 0.05, 0.1, 0.3, 0.5]
episodes = 10000

results = {}
history_results = {}
for alpha in alpha_values:
    env = GridWorld()
    agent = McAgent(epsilon=0.1, alpha=alpha)
    history = {'returns': [], 'q_delta': [], 'greedy_changes': []}
    snapshot_at = min(1000, episodes)
    q_snapshot = {}

    print(f'[epsilon = 0.1, alpha = {alpha}]    ')
    for episode in range(episodes):
        state = env.reset()
        agent.reset()
```

```
        #          MC
        while True:
            action = agent.get_action(state)
            next_state, reward, done = env.step(action)
            agent.add(state, action, reward)

            if done:
                G0, mean_q_delta, changes = agent.update()
                break
            state = next_state

        history['returns'].append(G0)
        history['q_delta'].append(mean_q_delta)
        history['greedy_changes'].append(changes)

        if episode + 1 == snapshot_at:
            q_snapshot = dict(agent.Q)

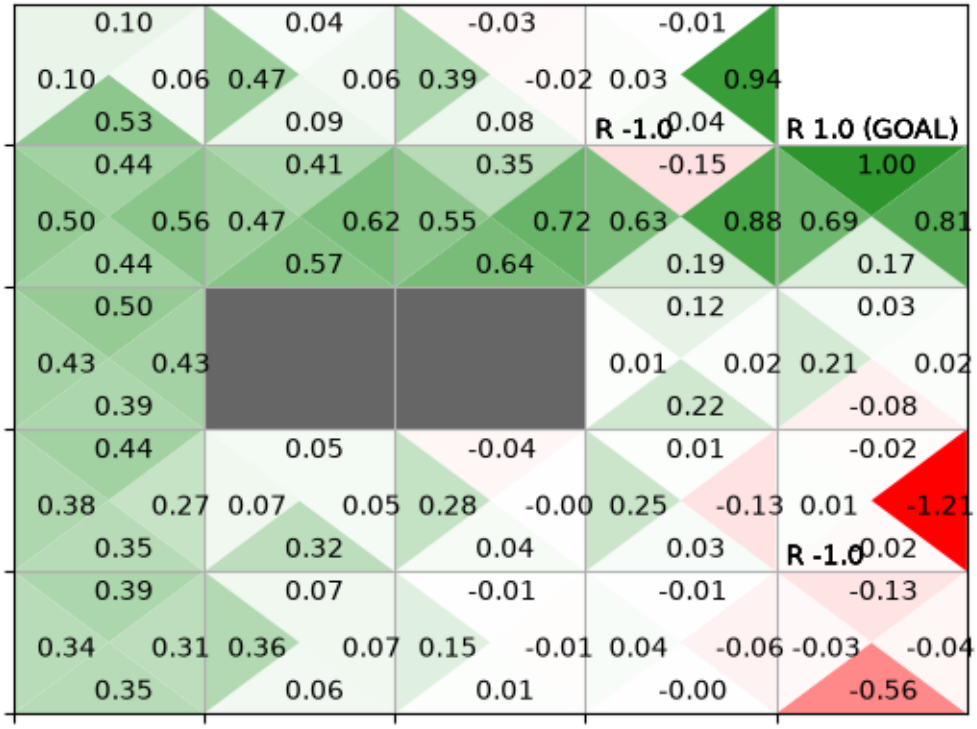
        if (episode + 1) % 1000 == 0 or episode + 1 == episodes:
            last = min(1000, episode + 1)
            print(f'Episode {episode + 1:5d}/{episodes}: '
                  f'  {last}    G0={np.mean(history["returns"][-last:]):.3f}, '
                  f'  |dQ|={np.mean(history["q_delta"][-last:]):.5f}, '
                  f'Greedy Action  ={sum(history["greedy_changes"][-last:])} ')

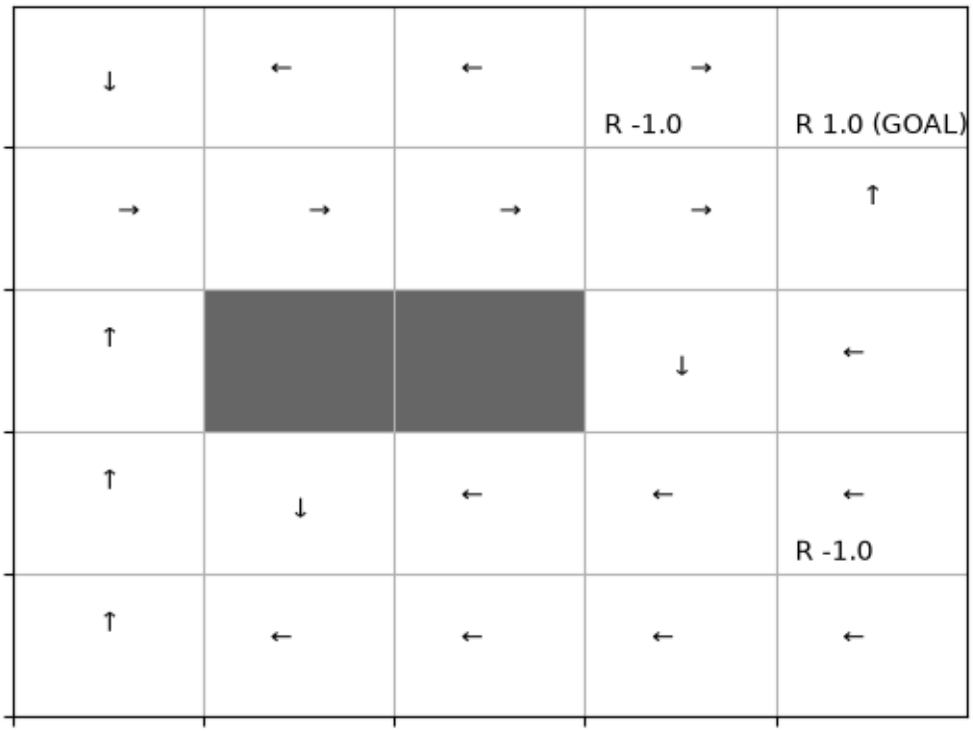
    avg_difference, n_pairs = summarize_q_change(q_snapshot, agent.Q)
    print(f'Q   ({snapshot_at} -> {episodes} Episode): '
          f'      ={avg_difference:.5f}, State-Action  ={n_pairs} ')
    results[alpha] = agent
    history_results[alpha] = history
    print(f'[epsilon = 0.1, alpha = {alpha}]     ')
    env.render_q(agent.Q)  #     Q   Greedy Policy

# alpha       : Return   Q
plot_learning_curves(history_results, 'alpha', alpha_values, window=200)
```

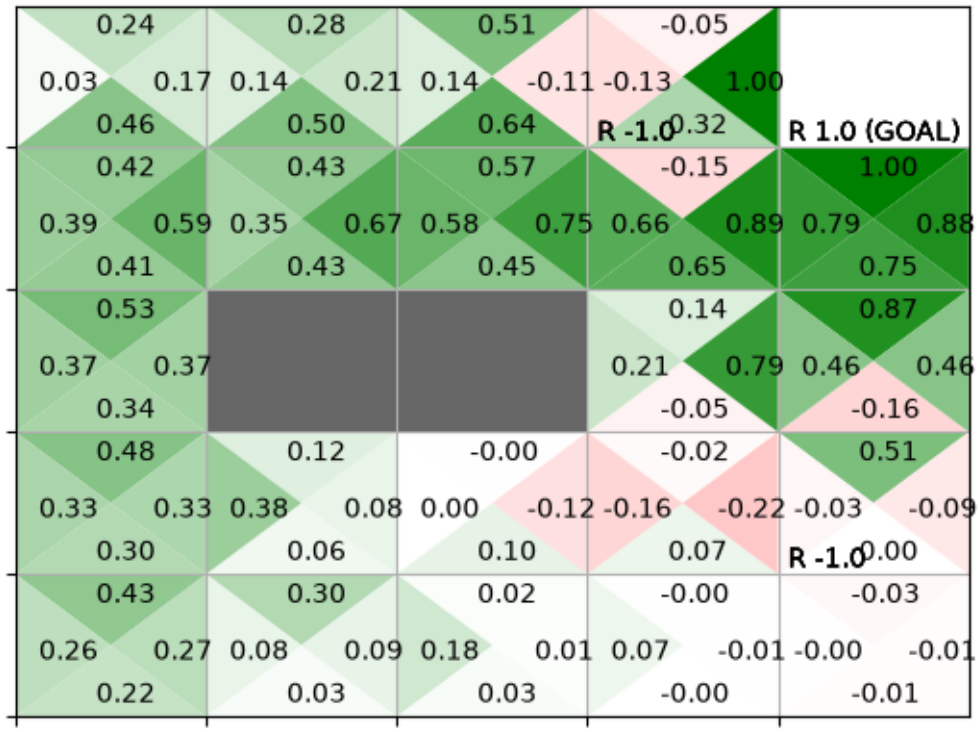
```
[epsilon = 0.1, alpha = 0.01]
Episode  1000/10000:   1000    G0=0.395,   |dQ|=0.00149, Greedy Action
 =123
Episode  2000/10000:   1000    G0=0.396,   |dQ|=0.00104, Greedy Action  =0
Episode  3000/10000:   1000    G0=0.397,   |dQ|=0.00096, Greedy Action  =0
Episode  4000/10000:   1000    G0=0.402,   |dQ|=0.00085, Greedy Action  =0
Episode  5000/10000:   1000    G0=0.399,   |dQ|=0.00083, Greedy Action  =0
Episode  6000/10000:   1000    G0=0.398,   |dQ|=0.00083, Greedy Action  =0
Episode  7000/10000:   1000    G0=0.390,   |dQ|=0.00091, Greedy Action  =0
Episode  8000/10000:   1000    G0=0.404,   |dQ|=0.00076, Greedy Action  =0
Episode  9000/10000:   1000    G0=0.403,   |dQ|=0.00077, Greedy Action  =0
```

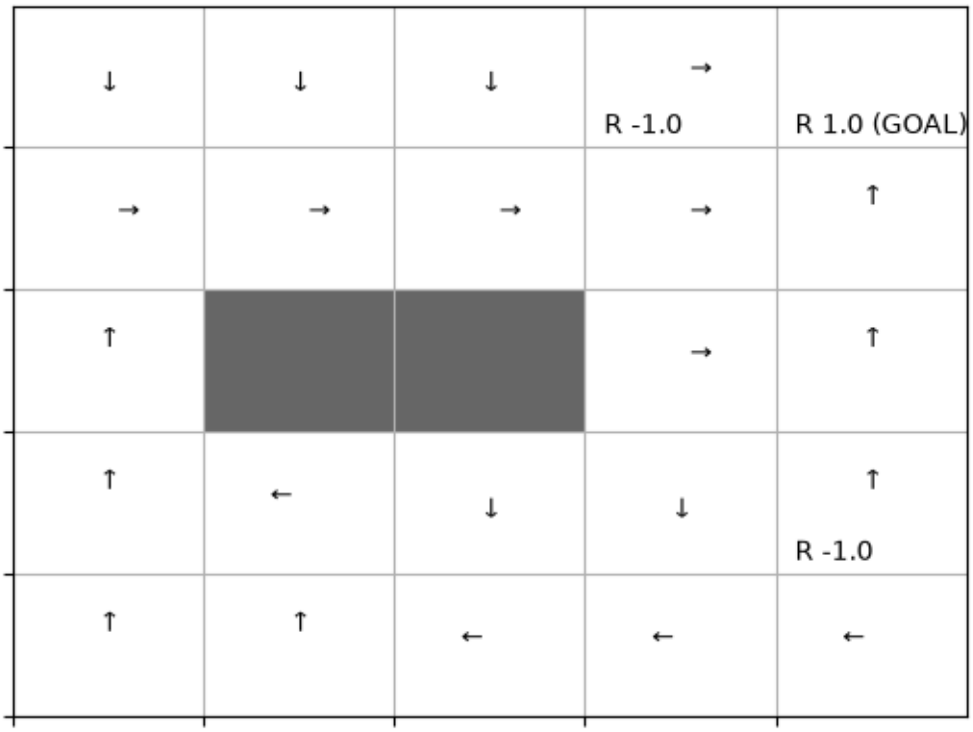
```
Episode 10000/10000:   1000   G0=0.399,   |dQ|=0.00078, Greedy Action  =0
Q   (1000 -> 10000 Episode):       =0.13248, State-Action  =88
[epsilon = 0.1, alpha = 0.01]
```



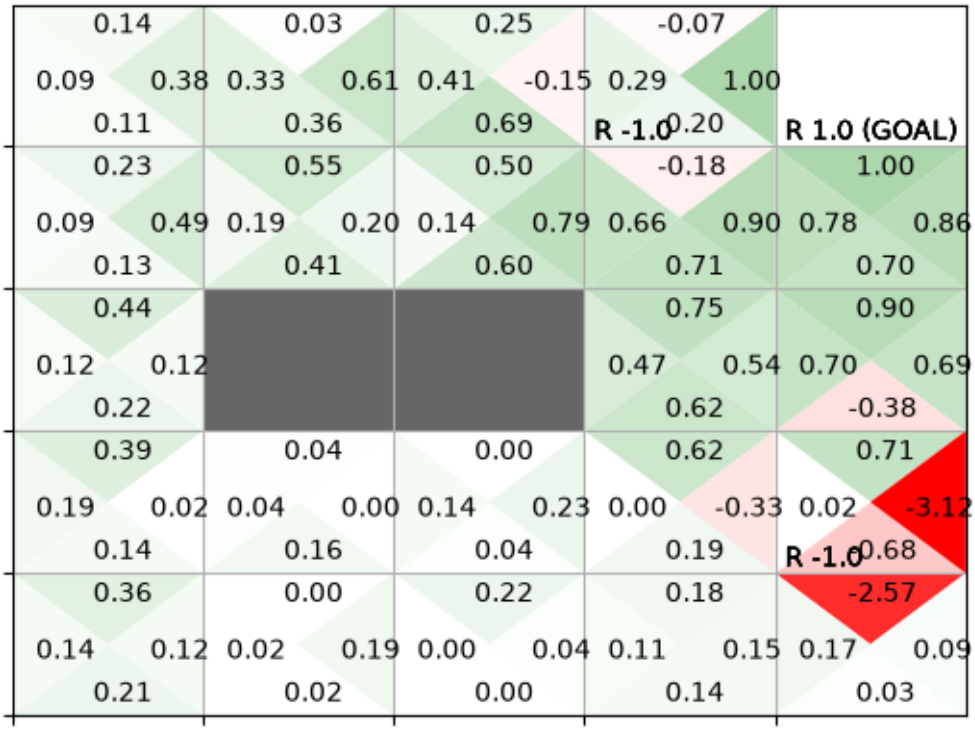


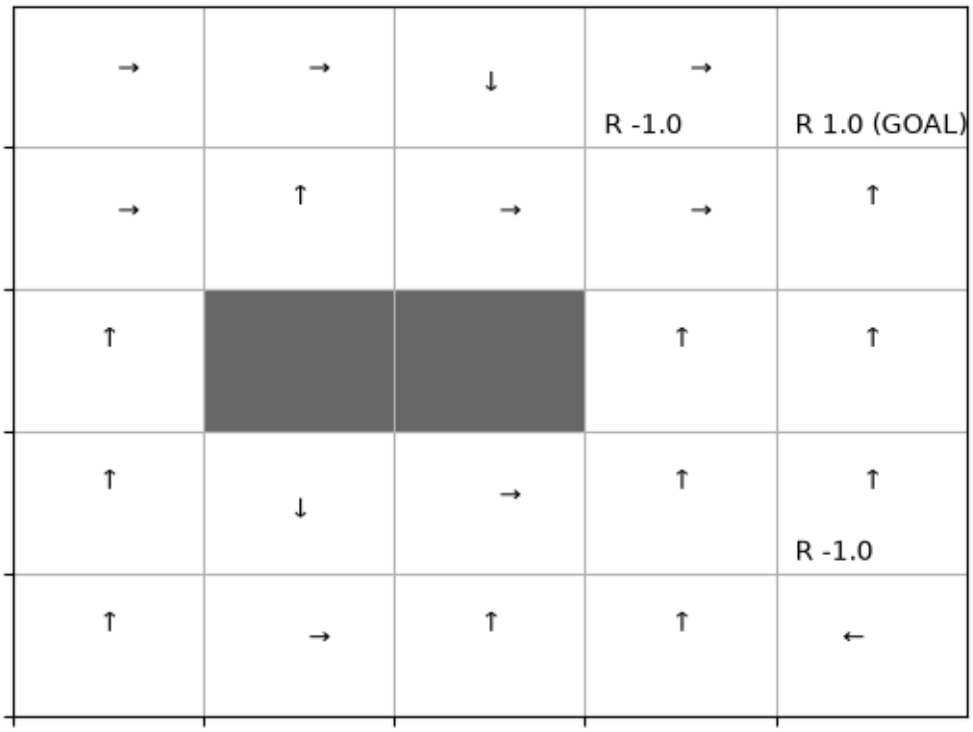

```
[epsilon = 0.1, alpha = 0.05]
Episode  1000/10000:   1000    G0=0.310,   |dQ|=0.00502, Greedy Action
 =135
Episode  2000/10000:   1000    G0=0.318,   |dQ|=0.00416, Greedy Action  =28
Episode  3000/10000:   1000    G0=0.370,   |dQ|=0.00362, Greedy Action  =5
Episode  4000/10000:   1000    G0=0.413,   |dQ|=0.00320, Greedy Action  =6
Episode  5000/10000:   1000    G0=0.409,   |dQ|=0.00342, Greedy Action  =21
Episode  6000/10000:   1000    G0=0.418,   |dQ|=0.00315, Greedy Action  =4
Episode  7000/10000:   1000    G0=0.417,   |dQ|=0.00305, Greedy Action  =15
Episode  8000/10000:   1000    G0=0.392,   |dQ|=0.00323, Greedy Action  =93
Episode  9000/10000:   1000    G0=0.323,   |dQ|=0.00381, Greedy Action  =13
Episode 10000/10000:   1000    G0=0.367,   |dQ|=0.00364, Greedy Action  =52
Q   (1000 -> 10000 Episode):       =0.12159, State-Action  =88
[epsilon = 0.1, alpha = 0.05]
```


0.24
0.28
0.51
-0.05
0.03
0.17
0.14
0.21
0.14
-0.11
-0.13
1.00
0.46
0.50
0.64
R -1.0
0.32
R 1.0 (GOAL)
0.42
0.43
0.57
-0.15
1.00
0.39
0.59
0.35
0.67
0.58
0.75
0.66
0.89
0.79
0.88
0.41
0.43
0.45
0.65
0.75
0.53
0.14
0.87
0.37
0.37
0.21
0.79
0.46
0.46
0.34
-0.05
-0.16
0.48
0.12
-0.00
-0.02
0.51
0.33
0.33
0.38
0.08
0.00
-0.12
-0.16
-0.22
-0.03
-0.09
0.30
0.06
0.10
0.07
R -1.0
0.00
0.43
0.30
0.02
-0.00
-0.03
0.26
0.27
0.08
0.09
0.18
0.01
0.07
-0.01
-0.00
-0.01
0.22
0.03
0.03
-0.00
-0.01

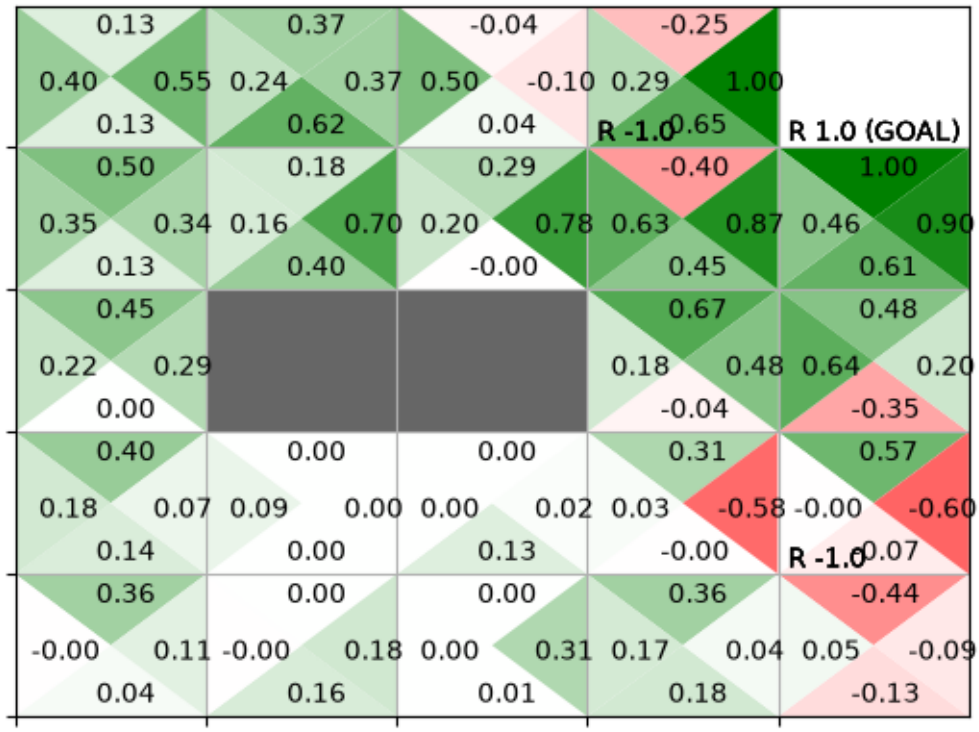

R -1.0
R 1.0 (GOAL)
R -1.0

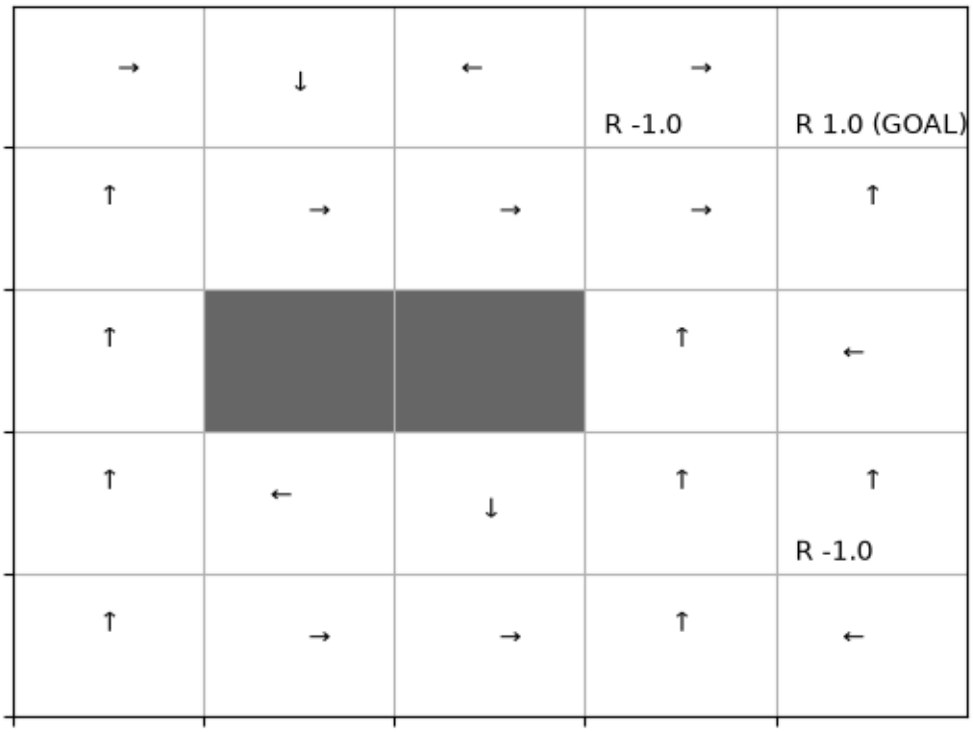
```
[epsilon = 0.1, alpha = 0.1]
Episode  1000/10000:   1000    G0=0.399,    |dQ|=0.00967, Greedy Action
 =191
Episode  2000/10000:   1000    G0=0.396,    |dQ|=0.00784, Greedy Action  =76
Episode  3000/10000:   1000    G0=0.401,    |dQ|=0.00857, Greedy Action  =35
Episode  4000/10000:   1000    G0=0.400,    |dQ|=0.00777, Greedy Action  =96
Episode  5000/10000:   1000    G0=0.372,    |dQ|=0.00818, Greedy Action  =72
Episode  6000/10000:   1000    G0=0.408,    |dQ|=0.00704, Greedy Action  =59
Episode  7000/10000:   1000    G0=0.409,    |dQ|=0.00704, Greedy Action  =55
Episode  8000/10000:   1000    G0=0.413,    |dQ|=0.00635, Greedy Action  =58
Episode  9000/10000:   1000    G0=0.409,    |dQ|=0.00707, Greedy Action  =89
Episode 10000/10000:   1000    G0=0.377,    |dQ|=0.00875, Greedy Action
 =121
Q   (1000 -> 10000 Episode):       =0.24016, State-Action  =88
[epsilon = 0.1, alpha = 0.1]
```



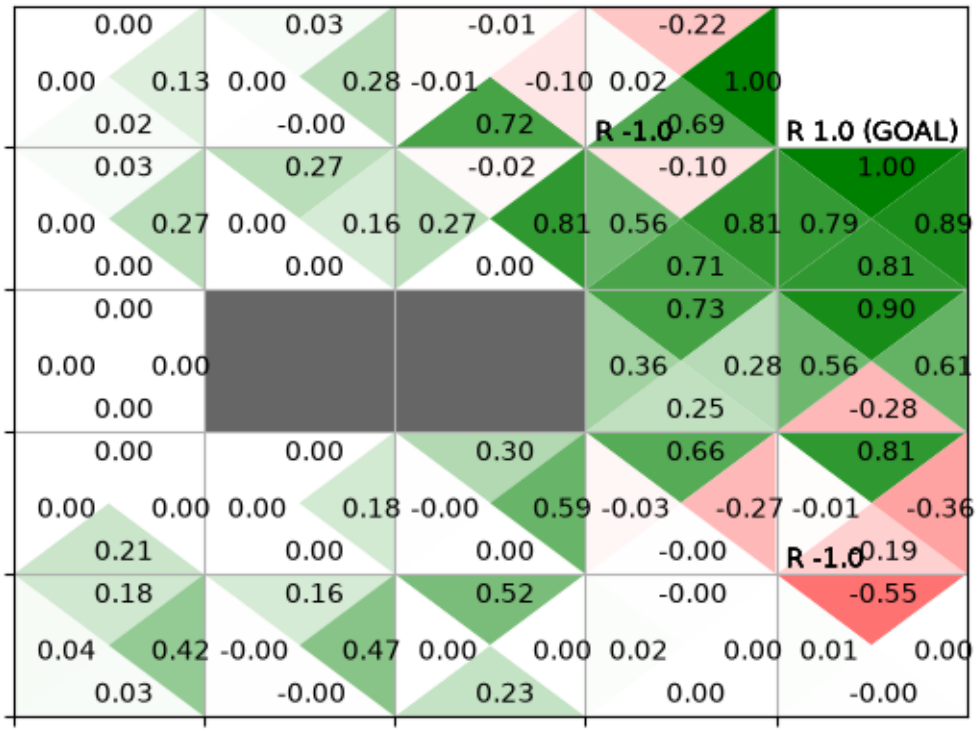


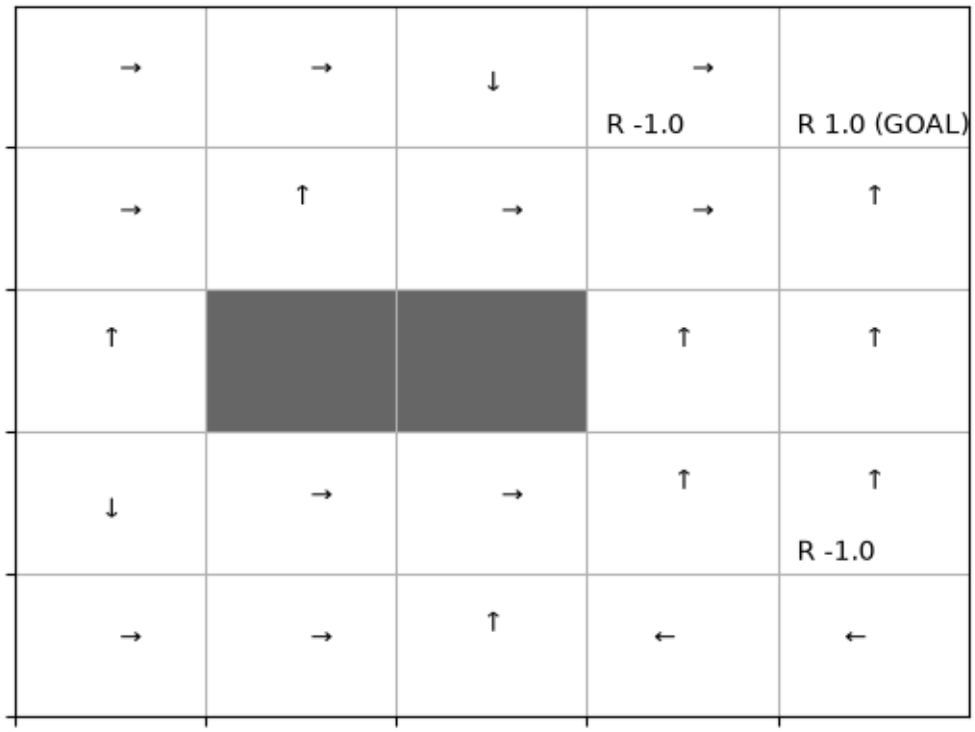

```
[epsilon = 0.1, alpha = 0.3]
Episode  1000/10000:   1000    G0=0.349,   |dQ|=0.02438, Greedy Action
 =747
Episode  2000/10000:   1000    G0=0.361,   |dQ|=0.02654, Greedy Action
 =556
Episode  3000/10000:   1000    G0=0.361,   |dQ|=0.02554, Greedy Action
 =619
Episode  4000/10000:   1000    G0=0.373,   |dQ|=0.02393, Greedy Action
 =509
Episode  5000/10000:   1000    G0=0.379,   |dQ|=0.02514, Greedy Action
 =481
Episode  6000/10000:   1000    G0=0.352,   |dQ|=0.02497, Greedy Action
 =585
Episode  7000/10000:   1000    G0=0.372,   |dQ|=0.02316, Greedy Action
 =533
Episode  8000/10000:   1000    G0=0.380,   |dQ|=0.02391, Greedy Action
 =440
Episode  9000/10000:   1000    G0=0.376,   |dQ|=0.02278, Greedy Action
 =527
Episode 10000/10000:   1000    G0=0.360,   |dQ|=0.02529, Greedy Action
 =723
Q   (1000 -> 10000 Episode):       =0.28851, State-Action  =88
[epsilon = 0.1, alpha = 0.3]
```

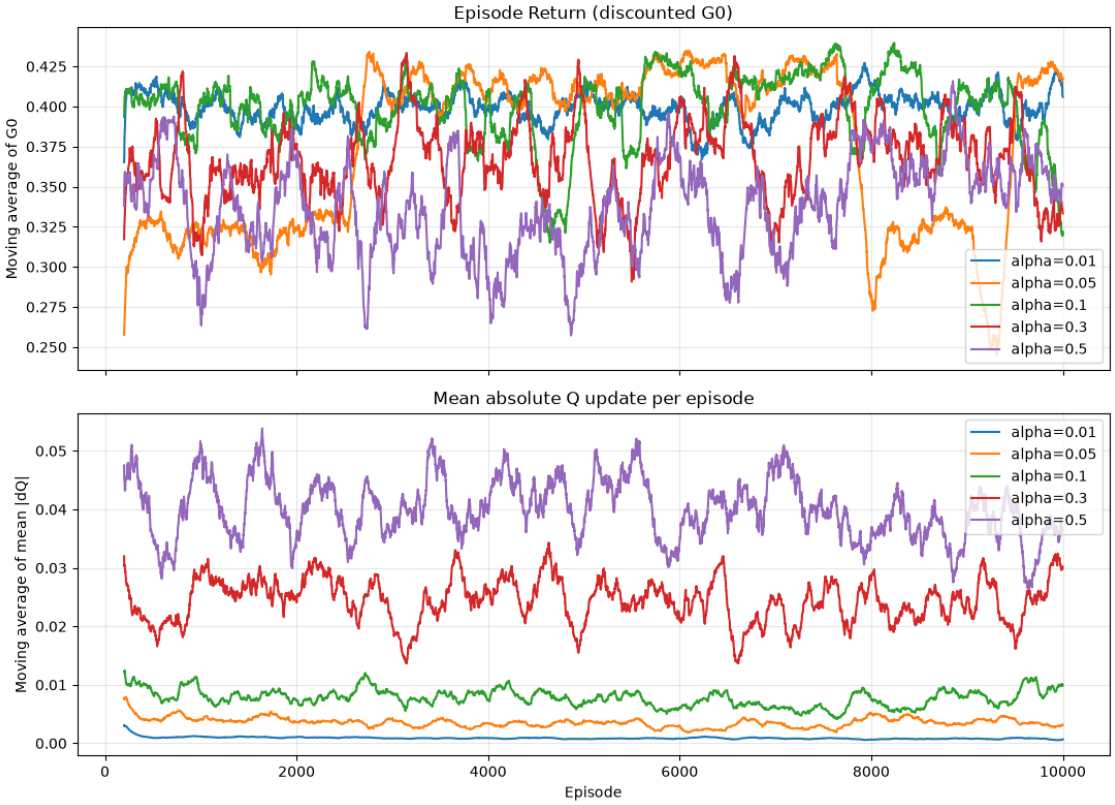

0.13
0.37
-0.04
-0.25
0.40
0.55
0.24
0.37
0.50
-0.10
0.29
1.00
0.13
0.62
0.04
R -1.0
0.65
R 1.0 (GOAL)
0.50
0.18
0.29
-0.40
1.00
0.35
0.34
0.16
0.70
0.20
0.78
0.63
0.87
0.46
0.90
0.13
0.40
-0.00
0.45
0.61
0.45
0.67
0.48
0.22
0.29
0.18
0.48
0.64
0.20
0.00
-0.04
-0.35
0.40
0.00
0.00
0.31
0.57
0.18
0.07
0.09
0.00
0.00
0.02
0.03
-0.58
-0.00
-0.60
0.14
0.00
0.13
-0.00
R -1.0
0.07
0.36
0.00
0.00
0.36
-0.44
-0.00
0.11
-0.00
0.18
0.00
0.31
0.17
0.04
0.05
-0.09
0.04
0.16
0.01
0.18
-0.13


R -1.0
R 1.0 (GOAL)
R -1.0

```
[epsilon = 0.1, alpha = 0.5]
Episode  1000/10000:   1000    G0=0.339,    |dQ|=0.04167, Greedy Action
 =1181
Episode  2000/10000:   1000    G0=0.342,    |dQ|=0.03992, Greedy Action
 =1427
Episode  3000/10000:   1000    G0=0.320,    |dQ|=0.03984, Greedy Action
 =1490
Episode  4000/10000:   1000    G0=0.322,    |dQ|=0.04201, Greedy Action
 =1934
Episode  5000/10000:   1000    G0=0.309,    |dQ|=0.04200, Greedy Action
 =1512
Episode  6000/10000:   1000    G0=0.343,    |dQ|=0.04167, Greedy Action
 =992
Episode  7000/10000:   1000    G0=0.324,    |dQ|=0.04141, Greedy Action
 =1667
Episode  8000/10000:   1000    G0=0.347,    |dQ|=0.03916, Greedy Action
 =1291
Episode  9000/10000:   1000    G0=0.368,    |dQ|=0.03655, Greedy Action
 =1006
Episode 10000/10000:   1000    G0=0.357,    |dQ|=0.03681, Greedy Action
 =1561
Q   (1000 -> 10000 Episode):       =0.19059, State-Action  =88
[epsilon = 0.1, alpha = 0.5]
```




R -1.0
R 1.0 (GOAL)
R -1.0


Episode Return (discounted G0)
Moving average of G0
0.425
0.400
0.375
0.350
0.325
0.300
0.275
0.250
alpha=0.01
alpha=0.05
alpha=0.1
alpha=0.3
alpha=0.5
Mean absolute Q update per episode
Moving average of mean |dQ|
0.05
0.04
0.03
0.02
0.01
0.00
0
2000
4000
6000
8000
10000
Episode